

ELEC 374 – Final Report

Ben Malvern, Clarke Needles, Jimmy Moutafis-Tymcio

March 3, 2026

“We do hereby verify that this written lab report is our own work and contains our own original ideas, concepts, and designs. No portion of this report has been copied in whole or in part from another source, with the possible exception of properly referenced material.”

Abstract

This report presents the design, implementation, and functional simulation of a Mini SRC (Simple RISC Computer) processor for the ELEC 374 laboratory project. The processor was developed in Verilog and features a three bus datapath architecture, which reduces instruction cycle counts by enabling parallel operand reads and writeback compared to a traditional single bus design. Arithmetic operations are supported through a Carry Lookahead Adder (CLA) and a Booth bit-pair recoding multiplier, among other Arithmetic Logic Unit (ALU) components. The control unit was implemented using a finite state machine approach (Method 1) and integrated with the datapath to automate instruction fetch, decode, and execution. The complete design was verified through functional simulation of the Phase 3 test program, which exercises all supported instructions including arithmetic, logical, load/store, branch, jump, subroutine call/return, and special instructions, with all register and memory values matching expected results. The project was completed up to Phase 3. FPGA implementation was not pursued.

Table of Contents

Abstract	2
Table of Contents	3
1. Project Specification.....	5
1.1 Project Objectives	5
1.2 Processor Architecture and System Components.....	5
1.3 Instruction Set and Addressing Modes	5
1.3.1 Overview of Instruction Set	5
1.3.2 Instruction Formats	6
1.3.3 Addressing Modes	6
2. Project Design and Implementation	7
2.1 Datapath	7
2.2 Control Unit	8
2.3 Supporting logic and Hardware Features	8
3. Evaluation Results	9
3.1 Maximum Frequency of Operation	9
3.2 Average Cycles Per Instruction	9
3.3 Chip Area Utilization.....	10
3.4 Additional Performance Metrics	11
4. Discussion.....	11
5. Conclusion and Future Work	12
References	12
Appendix A: Instruction Set.....	13
Appendix B: Functional Simulation Results	14
Appendix C: Verilog Code	17
Control Unit.....	17
Processor	27
Datapath	31
Select and Encode	38

Testbench.....	40
----------------	----

1. Project Specification

1.1 Project Objectives

The goal of this project is to design and implement a Simple RISC Computer (Mini SRC). The Mini SRC consists of a processor datapath, control unit, and memory interface will follow architectural principles of the SRC as shown in course material.

This project focuses on a complete development workflow. The processor is implemented using Verilog modules, while simulation and validation is done through ModelSim via testbenches also written in Verilog. Validation for instructions are done by manually analyzing the waveforms in the simulation environment.

This project reinforces computer architecture concepts, including RISC processor design, instruction fetch, decoding, and execution, as well as datapath interactions with the control unit and memory system. Overall, this project aims to develop a functional processor while gaining a deep understanding of how individual hardware components work together within the Mini SRC architecture.

1.2 Processor Architecture and System Components

The Mini SRC is 32-bit processor with a 32-bit datapath and sixteen 32-bit registers. Registers R0-R7 are general purpose registers, R8-R11 are argument registers, R12 is the return address register (RA), R13-R14 are return value registers, and R15 is the stack pointer (SP). There are two additional dedicated 32-bit registers HI and LO. The HI register stores the high-order bits of multiplication result or the remainder of a division operation. The LO register stores the low-order bits of multiplication result or the quotient of a division operation. The final two registers are the 32-bit Program Counter (PC) and Instruction Register (IR). The Mini SRC contains an ALU that performs all arithmetic and logical operations, and shift/rotations. The memory subsystem consists of Random Access Memory (RAM) that is 512 words, the Memory Address Register (MAR) which holds the memory address, and the Memory Data Register (MDR) that holds data being read/written. There are also 32-bit external input and output ports.

1.3 Instruction Set and Addressing Modes

1.3.1 Overview of Instruction Set

The Mini SRC uses a 32-bit format for each instruction. The instruction set is categorized into load/store, arithmetic and logical, conditional branch and jump, input/output, and miscellaneous instructions.

1.3.2 Instruction Formats

The instructions can be categorized into the following formats as shown below.

Name	Fields					Comments
Field size	31..27 5 bits	26..23 4 bits	22..19 4 bits	18..15 4 bits	14..0 15 bits	All instructions are 32-bit long
R-Format	OP-code	Ra	Rb	Rc	Unused	Arithmetic/Logical
I-Format	OP-code	Ra	Rb	Constant C / Unused		Arithmetic/Logical; Load/Store; Imm.
B-Format	OP-code	Ra	C2	Constant C		Branch
J-Format	OP-code	Ra	Unused			Jump; Input/Output; Special
M-Format	OP-code	Unused				Misc.

Table 1: Instruction Formats

The opcode for the instructions as well as the instruction description and usage can be seen in Appendix A: Instruction Set.

1.3.3 Addressing Modes

1.3.3.1 Direct Addressing

The instruction contains the memory address of the operand directly. Example: load the value stored at memory address 100 into R1.

```
ld R1, 100
```

1.3.3.2 Indexed Addressing

The effective address is formed by adding a base register and an offset. Example: load from address R2 + 20 into R1

```
ld R1, 20(R2)
```

1.3.3.3 Register Addressing

Operands are located within registers. Example: simple addition instruction.

```
add R1, R2, R3
```

1.3.3.4 Register Indirect Addressing

A register contains the memory address of the operand. Example: load from memory address stored in R2 into R1.

```
ld R1, 0(R2)
```

1.3.3.5 Immediate Addressing

The operand is a constant value in the instruction. Example: load constant 5 into R1.

```
ldi R1, 5
```

1.3.3.6 Relative Addressing

The effective address is computed with PC and offset. Example: branch to instruction at PC + 20.

```
brzr R1, 20
```

2. Project Design and Implementation

2.1 Datapath

The Datapath is responsible for storing, moving, and transforming data between the subsystems during instruction execution. The Datapath was designed around the Mini SRC hardware components including the general-purpose registers (R0-R15), the PC, IR, MAR, MDR, and the special purpose HI and LO registers used for multiplication and division results. Each register holds 32 bits of data.

A major design choice was the use of a three-bus Datapath. Bus A and B provide the two operands required by the ALU, and Bus C is used to return results to the designated register. This allowed the processor to move multiple operands with fewer temporary transfers required, which allowed for fewer clock cycles to perform an instruction. This design makes arithmetic, logical, and address-computation instructions more efficient.

The ALU was designed to support the required set of operations for the mini SRC Processor. These operations include addition, subtraction, multiplication, division, logical AND, OR negate, NOT, shift right, shift right arithmetic, shift left, rotate right and rotate left. The ALU received input from Bus A and B, and would output onto bus C. In the final implementation, addition and subtraction were done using a CLA so that carry signals could be generated more efficiently. For multiplication, Booth bit-pair recoding was used, which reduced the number of partial products compared to the more basic shift and add multiplier. For division, the non-restoring division algorithm was used. The outputs of multiplication and division are stored across HI and LO, which allows the processor to support mfhi and mflo instructions.

The Datapath also uses a memory interface for load and store operations. The MAR holds the memory address currently being used, and the MDR stores the data that was either

read from or will be written to memory, used like an interface to the RAM. The RAM was implemented as byte-addressable, little-endian memory.

2.2 Control Unit

The control unit allows the processor to function as a CPU. Our final design uses a finite state machine (FSM) control unit to automatically generate the control signals required for fetch, decode, execution, memory access and writeback.

The control unit is organized as a sequence of states. The finite state machine iterates through each sequence of states based on the decoded opcode that was read from memory. After the instruction is decoded, the control sequence begins through the common fetch cycle states first, then goes down the instruction-specific sequence. This is where the three bus designs have clear advantages.

Since the Datapath uses a three-bus structure, the control unit benefits from a more efficient execution model. Two operands can be read in parallel while the result is written back on a separate bus, allowing many instructions to be sequenced in fewer steps than the single bus would allow. Overall, this makes the architecture of the control unit cleaner and faster.

One of the most important hardware features was the select and encode logic within the Control Unit. This module decodes the register fields from the instruction and generates the control signals needed to activate the designated source and destination registers. This allows the control unit to activate the source and destination registers using Gra, Grb, and Grc control signals, and place them on their designated bus. This is what allows the instructions to execute using programmer specified sources and destinations. The select and encode logic also produces the sign-extended immediate value used by the immediate type instructions.

2.3 Supporting logic and Hardware Features

The CON FF logic is used for conditional branching. The Mini SRC design doesn't rely on a separate condition code register, so the branch decisions are made by directly evaluating a register value. The module evaluates the condition, then stores the result in a flip flop to indicate if it is true or not, determining if that branch should be taken. This is necessary for branch instructions such as brzr, brnz, brpl, and brmi.

The processor also includes dedicated input and output ports to support in and out instructions required by the Mini SRC architecture. These ports serve as an interface for external interaction with the CPU. Including these ports in the design enables the possibility of developing the user interface.

3. Evaluation Results

3.1 Maximum Frequency of Operation

The processor was evaluated using a fixed clock period of 10 ns for the simulations. This corresponds to a maximum operating frequency of 100 MHz. This clock period was chosen for stable testing and easy analysis for instruction execution.

This frequency only reflects simulation and does not consider any hardware limits. For physical implementation, the maximum operating frequency would be determined by various factors. The datapath complexity and most importantly propagation delay for the combinational logic are the two key players.

3.2 Average Cycles Per Instruction

The Cycles Per Instruction (CPI) of each instruction can be determined from the control unit code FSM directly. Each instruction takes three cycles to fetch the instruction and then a variable number of other cycles depending on the instruction type. The average CPI is calculated as the total cycles divided by the number of instructions. All the arithmetic and logical instructions took 5 cycles except for mul and div which took 6 cycles, and mfhi and mflo which took 4 cycles. The ld and st instructions took 7 cycles, while the ldi instruction only took 5 cycles. The br instruction took 6 cycles, the jal instruction took 5, and the jr instruction took 4. The I/O instructions took 4 cycles each. The nop takes 3 cycles, and the halt instruction takes 4. The average CPI for each instruction type and overall CPI is shown in the table below.

Table 2: Average CPI

Group	Average CPI
Arithmetic/Logical	$90/18 = 5.00$
Load/Store	$19/3 = 6.33$
Branch/Jump	$15/3 = 5.00$
I/O	$8/2 = 4.00$
Miscellaneous	$7/2 = 3.50$
Overall	$139/28 = 4.96$

3.3 Chip Area Utilization

The design was synthesized using Quartus and rather than using an FPGA, the synthesis and elaboration was targeting a Cyclone V FPGA (5CGXFC7C7F23C8). The compilation results are shown below.

Fitter Status	Successful - Fri Apr 03 23:34:18 2026
Quartus Prime Version	18.1.1 Build 646 04/11/2019 SJ Lite Edition
Revision Name	ELEC374_CPU
Top-level Entity Name	processor
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	10,375 / 56,480 (18 %)
Total registers	17259
Total pins	68 / 268 (25 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total RAM Blocks	0 / 686 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1: Compilation Report

The key insights are that the processor utilizes approximately 18% of the Available Logic Elements (ALMs). This relatively large amount of the FPGA's logic resources are being used because the RAM is being implemented using ALMs instead of FPGA memory. This is clear because of the 0% memory utilization in the previous figure. The use of the three-bus architecture increases hardware requirements due to additional multiplexing and routing logic. However, given these fallbacks, the extra buses improve instruction throughput. In contrast to the simple single-bus design there is a clear tradeoff between performance and hardware resources.

3.4 Additional Performance Metrics

Comparing the CPI to the single-bus sequence, we can see how much better the three-bus approach is. The key improvements are that the single-bus sequence needs a Y register, ls/st need separate address and data movement steps, and jal takes an extra cycle. The improvements are shown on the table below.

Figure 2: CPI Improvements

Group	Single-Bus CPI	Improvement
Arithmetic/Logical	$104/18 = 5.78$	0.78 cycles lower (13.5%)
Load/Store	$22/3 = 7.33$	1.00 cycle lower (13.6%)
Branch/Jump	$17/3 = 5.67$	0.67 cycles lower (11.8%)
I/O	$8/2 = 4.00$	No change
Miscellaneous	$7/2 = 3.50$	No change
Overall	$158/28 = 5.64$	0.68 cycles lower (12.1%)

4. Discussion

The biggest challenge in this project was transitioning from the single bus datapath in Phase 1 to the three-bus architecture in Phase 2. In the single bus design, the ALU received one operand from the Y register and the other from the bus, meaning each arithmetic instruction needed a dedicated cycle just to stage a value into Y. The three-bus design removed this by feeding operands directly from Bus A and Bus B into the ALU, eliminating the Y register from the final design.

The three-bus architecture introduced additional complexity in the select and encode logic, which had to be expanded to support independent register selection for each bus. In Phase 3, separate Gra, Grb, and Grc signals were added for each bus, allowing different register fields to drive Bus A, Bus B, and Bus C at the same time. This made the control unit more flexible but increased the number of control signals it needed to manage.

The control unit was implemented using Method 1, the state machine approach, because it is easier to read and extend with new instructions compared to Method 2. All testing was done through functional simulation in ModelSim. While simulation confirmed correctness for Phase 3, it did not account for timing constraints or resource limitations that would come up during FPGA synthesis. Ideally, the design would have been synthesized and tested on hardware in Phase 4 to identify any issues that simulation might not catch.

5. Conclusion and Future Work

This project successfully designed and verified a Mini SRC processor in Verilog, completed up to Phase 3. The final design uses a three-bus datapath with a carry lookahead adder, Booth bit-pair recoding multiplier, and non-restoring division unit, all controlled by a finite state machine control unit. The Phase 3 test program ran successfully in simulation, with all register values and memory contents matching the expected results.

A key decision in this project was switching from a single bus to a three bus datapath in Phase 2. This reduced the number of clock cycles per instruction by allowing two operands to be read and a result written back at the same time. In Phase 3, Bus C was upgraded to allow registers to drive it directly, which removed extra steps that were previously needed for instructions like jump and link.

If this project were continued, the next step would be completing Phase 4 by putting the design onto an FPGA and testing it on hardware. After that, pipelining could be added to improve performance. Other possible extensions include interrupt handling, new instructions, or additional addressing modes.

References

Department of Electrical and Computer Engineering. (2026). *Designing a Simple RISC Computer: CPU Specification*. Kingston, ON: Queen's University.

Appendix A: Instruction Set

Table 3: Instruction Set

Instruction	Opcode	Format	Description
add	00000	R-Format	$R[Ra] \leftarrow R[Rb] + R[Rc]$
sub	00001	R-Format	$R[Ra] \leftarrow R[Rb] - R[Rc]$
and	00010	R-Format	$R[Ra] \leftarrow R[Rb] \wedge R[Rc]$
or	00011	R-Format	$R[Ra] \leftarrow R[Rb] \vee R[Rc]$
shr	00100	R-Format	Shift right $R[Rb]$ into $R[Ra]$ by count in $R[Rc]$
shra	00101	R-Format	Shift right arithmetic $R[Rb]$ into $R[Ra]$ by count in $R[Rc]$
shl	00110	R-Format	Shift left $R[Rb]$ into $R[Ra]$ by count in $R[Rc]$
ror	00111	R-Format	Rotate right $R[Rb]$ into $R[Ra]$ by count in $R[Rc]$
rol	01000	R-Format	Rotate left $R[Rb]$ into $R[Ra]$ by count in $R[Rc]$
addi	01001	I-Format	$R[Ra] \leftarrow R[Rb] + C$ (sign-extended)
andi	01010	I-Format	$R[Ra] \leftarrow R[Rb] \wedge C$ (sign-extended)
ori	01011	I-Format	$R[Ra] \leftarrow R[Rb] \vee C$ (sign-extended)
div	01100	I-Format	$HI, LO \leftarrow R[Ra] \div R[Rb]$
mul	01101	I-Format	$HI, LO \leftarrow R[Ra] \times R[Rb]$
neg	01110	I-Format	$R[Ra] \leftarrow -R[Rb]$
not	01111	I-Format	$R[Ra] \leftarrow \neg(R[Rb])$
ld	10000	I-Format	$R[Ra] \leftarrow M[R[Rb] + C]$ (sign-extended)
ldi	10001	I-Format	$R[Ra] \leftarrow R[Rb] + C$ (sign-extended)
st	10010	I-Format	$M[R[Rb] + C]$ (sign-extended) $\leftarrow R[Ra]$
jal	10011	J-Format	Jump and link: $R[12] \leftarrow PC + 1$
jr	10100	J-Format	Return from procedure: $PC \leftarrow R[Ra]$
branch	10101	B-Format	$PC \leftarrow PC + 1 + C$ (sign-extended), if $R[Ra]$ meets the condition (--00 for branch if zero, --01 for branch if nonzero, --10 for branch if positive, --11 for branch if negative)
in	10110	J-Format	$R[Ra] \leftarrow \text{In.Port}$
out	10111	J-Format	$\text{Out.Port} \leftarrow R[Ra]$
mfhi	11000	J-Format	Move from HI: $R[Ra] \leftarrow HI$
mflo	11001	J-Format	Move from LO: $R[Ra] \leftarrow LO$
nop	11010	M-Format	No-operation: do nothing
halt	11011	M-Format	Halt the control stepping process

Appendix B: Functional Simulation Results

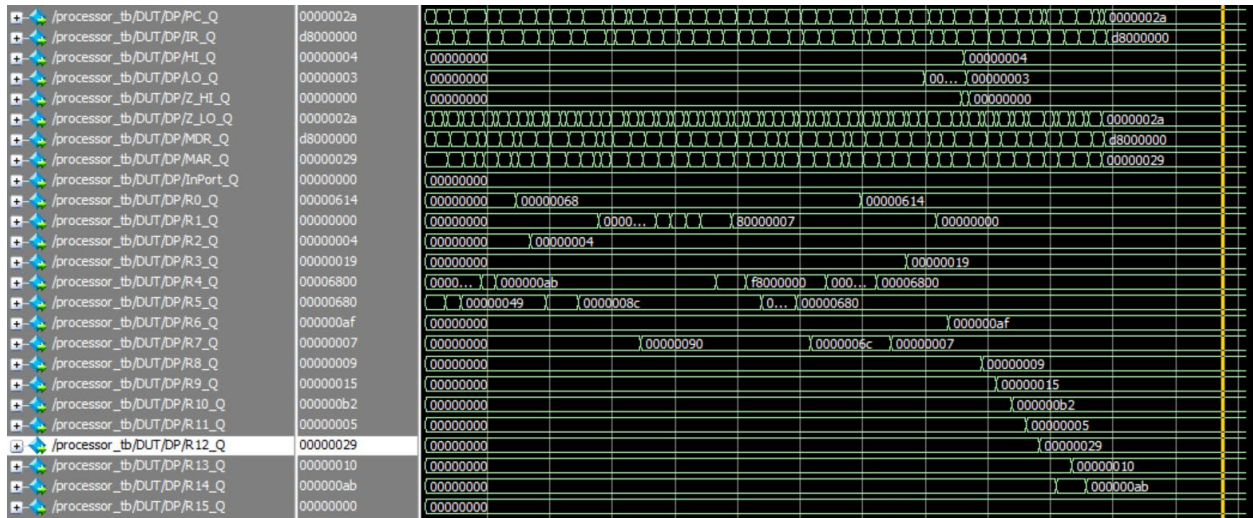


Figure 3: Full waveform of Phase 3 program execution

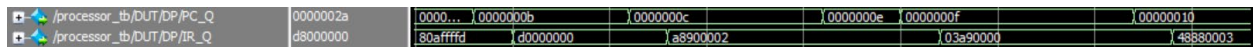


Figure 4: PC being incremented and IR changing as instructions are fetched

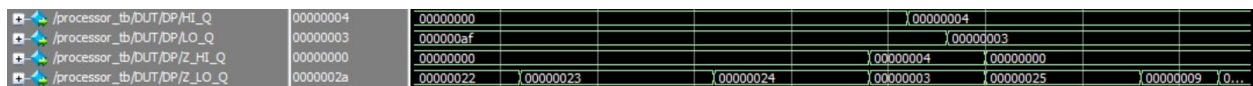


Figure 5: Hi and Lo changing after div instruction

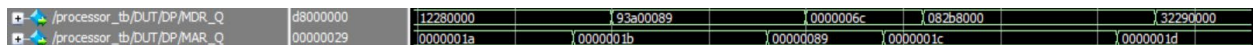


Figure 6: MDR and MAR waveforms for a brief period


```

# =====
# MEMORY BEFORE EXECUTION
# =====
# MEM[89] = 000000a7
# MEM[A3] = 00000068
# =====
# === MEMORY at 4000ns ===
# MEM[89] = 0000006c
# MEM[A3] = 00000008
# =====
# PHASE 3 PROGRAM COMPLETE
# =====
# PC    = 0000002a
# IR    = d8000000
# MAR   = 00000029
# MDR   = d8000000
# HI    = 00000004
# LO    = 00000003
# R0    = 00000614
# R1    = 00000000
# R2    = 00000004
# R3    = 00000019
# R4    = 00006800
# R5    = 00000680
# R6    = 000000af
# R7    = 00000007
# R8    = 00000009
# R9    = 00000015
# R10   = 000000b2
# R11   = 00000005
# R12   = 00000029
# R13   = 00000010
# R14   = 000000ab
# R15   = 00000000
# MEM[89] = 0000006c
# MEM[A3] = 00000008
# =====
# ** Note: $stop      : C:/Users/jimst/ELEC374_CPU/Phase 3/processor_tb.v(164)
#    Time: 6570 ns  Iteration: 0  Instance: /processor_tb

```

Figure 11: ModelSim transcript showing memory contents before and after execution, and final register values

Appendix C: Verilog Code

Control Unit

```
`timescale 1ns/10ps

module control_unit (
    input wire Clock,
    input wire Reset,
    input wire Stop,
    input wire CON_FF,
    input wire [31:0] IR,

    output reg Run,
    output reg Clear,

    // Select / encode signals
    output reg GraA,
    output reg GrbA,
    output reg GrcA,
    output reg GraB,
    output reg GrbB,
    output reg GrcB,
    output reg GraC,
    output reg GrbC,
    output reg GrcC,
    output reg GraIn,
    output reg GrbIn,
    output reg GrcIn,
    output reg Rin,
    output reg Rl2in,
    output reg RoutA,
    output reg RoutB,
    output reg RoutC,
    output reg BAout,

    // Datapath register control
    output reg PCin,
    output reg IRin,
    output reg MARin,
    output reg MDRin,
    output reg Zin,
    output reg HIin,
    output reg LOin,
    output reg Cin,

    // Memory / I/O control
    output reg Read,
    output reg Write,
    output reg OutPortin,

    // Special control
    output reg IncPC,
    output reg CONin,

    // ALU control
    output reg ADD,
    output reg SUB,
    output reg AND_op,
    output reg OR_op,
```

```

output reg SHR_op,
output reg SHRA_op,
output reg SHL_op,
output reg ROR_op,
output reg ROL_op,
output reg NEG_op,
output reg NOT_op,
output reg MUL_op,
output reg DIV_op,

// A bus drivers
output reg PCoutA,
output reg HIoutA,
output reg LOoutA,
output reg ZhighoutA,
output reg ZlowoutA,
output reg MDRoutA,
output reg InPortoutA,
output reg CoutA,

// B bus drivers
output reg PCoutB,
output reg HIoutB,
output reg LOoutB,
output reg ZhighoutB,
output reg ZlowoutB,
output reg MDRoutB,
output reg InPortoutB,
output reg CoutB,

// C bus drivers
output reg PCoutC,
output reg HIoutC,
output reg LOoutC,
output reg ZhighoutC,
output reg ZlowoutC,
output reg MDRoutC,
output reg InPortoutC,
output reg CoutC
);

```

```

localparam [4:0]
    OP_ADD    = 5'b00000,
    OP_SUB    = 5'b00001,
    OP_AND    = 5'b00010,
    OP_OR     = 5'b00011,
    OP_SHR    = 5'b00100,
    OP_SHRA   = 5'b00101,
    OP_SHL    = 5'b00110,
    OP_ROR    = 5'b00111,
    OP_ROL    = 5'b01000,
    OP_ADDI   = 5'b01001,
    OP_ANDI   = 5'b01010,
    OP_ORI    = 5'b01011,
    OP_DIV    = 5'b01100,
    OP_MUL    = 5'b01101,
    OP_NEG    = 5'b01110,
    OP_NOT    = 5'b01111,
    OP_LD     = 5'b10000,
    OP_LDI    = 5'b10001,
    OP_ST     = 5'b10010,

```

```

OP_JAL    = 5'b10011,
OP_JR     = 5'b10100,
OP_BR     = 5'b10101,
OP_IN     = 5'b10110,
OP_OUT    = 5'b10111,
OP_MFHI   = 5'b11000,
OP_MFLO   = 5'b11001,
OP_NOP    = 5'b11010,
OP_HALT   = 5'b11011;

```

```

localparam [6:0]

```

```

S_RESET   = 7'd0,
S_FETCH0  = 7'd1,
S_FETCH1  = 7'd2,
S_FETCH2  = 7'd3,
S_DECODE  = 7'd4,

S_RR3     = 7'd5,
S_RR4     = 7'd6,

S_IMM3    = 7'd7,
S_IMM4    = 7'd8,

S_UN3     = 7'd9,
S_UN4     = 7'd10,

S_LD3     = 7'd11,
S_LD4     = 7'd12,
S_LD5     = 7'd13,
S_LD6     = 7'd14,

S_ST3     = 7'd15,
S_ST4     = 7'd16,
S_ST5     = 7'd17,
S_ST6     = 7'd18,

S_BRANCH3 = 7'd19,
S_BRANCH4 = 7'd20,
S_BRANCH5 = 7'd21,

S_JR3     = 7'd22,

S_JAL3    = 7'd23,
S_JAL4    = 7'd24,

S_MUL3    = 7'd25,
S_MUL4    = 7'd26,
S_MUL5    = 7'd27,

S_DIV3    = 7'd28,
S_DIV4    = 7'd29,
S_DIV5    = 7'd30,

S_MFHI3   = 7'd31,
S_MFLO3   = 7'd32,

S_IN3     = 7'd33,
S_OUT3    = 7'd34,

S_HALT    = 7'd63;

```

```

reg [6:0] present_state, next_state;
wire [4:0] opcode = IR[31:27];

always @(posedge Clock or posedge Reset) begin
    if (Reset)
        present_state <= S_RESET;
    else
        present_state <= next_state;
end

always @(*) begin
    next_state = present_state;

    if (Stop) begin
        next_state = S_HALT;
    end else begin
        case (present_state)
            S_RESET:    next_state = S_FETCH0;
            S_FETCH0:   next_state = S_FETCH1;
            S_FETCH1:   next_state = S_FETCH2;
            S_FETCH2:   next_state = S_DECODE;

            S_DECODE: begin
                case (opcode)
                    OP_ADD, OP_SUB, OP_AND, OP_OR, OP_SHR, OP_SHRA, OP_SHL, OP_ROR, OP_ROL:
                        next_state = S_RR3;

                    OP_ADDI, OP_ANDI, OP_ORI, OP_LDI:
                        next_state = S_IMM3;

                    OP_NEG, OP_NOT:
                        next_state = S_UN3;

                    OP_LD:
                        next_state = S_LD3;

                    OP_ST:
                        next_state = S_ST3;

                    OP_BR:
                        next_state = S_BRANCH3;

                    OP_JR:
                        next_state = S_JR3;

                    OP_JAL:
                        next_state = S_JAL3;

                    OP_MUL:
                        next_state = S_MUL3;

                    OP_DIV:
                        next_state = S_DIV3;

                    OP_MFHI:
                        next_state = S_MFHI3;

                    OP_MFLO:
                        next_state = S_MFLO3;

                    OP_IN:

```

```

        next_state = S_IN3;

    OP_OUT:
        next_state = S_OUT3;

    OP_NOP:
        next_state = S_FETCH0;

    OP_HALT:
        next_state = S_HALT;

    default:
        next_state = S_FETCH0;
endcase
end

S_RR3:    next_state = S_RR4;
S_RR4:    next_state = S_FETCH0;

S_IMM3:   next_state = S_IMM4;
S_IMM4:   next_state = S_FETCH0;

S_UN3:    next_state = S_UN4;
S_UN4:    next_state = S_FETCH0;

S_LD3:    next_state = S_LD4;
S_LD4:    next_state = S_LD5;
S_LD5:    next_state = S_LD6;
S_LD6:    next_state = S_FETCH0;

S_ST3:    next_state = S_ST4;
S_ST4:    next_state = S_ST5;
S_ST5:    next_state = S_ST6;
S_ST6:    next_state = S_FETCH0;

S_BRANCH3: next_state = S_BRANCH4;
S_BRANCH4: next_state = S_BRANCH5;
S_BRANCH5: next_state = S_FETCH0;

S_JR3:    next_state = S_FETCH0;

S_JAL3:    next_state = S_JAL4;
S_JAL4:    next_state = S_FETCH0;

S_MUL3:    next_state = S_MUL4;
S_MUL4:    next_state = S_MUL5;
S_MUL5:    next_state = S_FETCH0;

S_DIV3:    next_state = S_DIV4;
S_DIV4:    next_state = S_DIV5;
S_DIV5:    next_state = S_FETCH0;

S_MFHI3:   next_state = S_FETCH0;
S_MFLO3:   next_state = S_FETCH0;

S_IN3:     next_state = S_FETCH0;
S_OUT3:    next_state = S_FETCH0;

S_HALT:    next_state = S_HALT;

default:   next_state = S_FETCH0;

```

```

        endcase
    end
end

always @(*) begin
    Run    = 1'b1;
    Clear  = 1'b0;

    GraA = 1'b0; GrbA = 1'b0; GrcA = 1'b0;
    GraB = 1'b0; GrbB = 1'b0; GrcB = 1'b0;
    GraC = 1'b0; GrbC = 1'b0; GrcC = 1'b0;
    GraIn = 1'b0; GrbIn = 1'b0; GrcIn = 1'b0;
    Rin = 1'b0; Rl2in = 1'b0;
    RoutA = 1'b0; RoutB = 1'b0; RoutC = 1'b0; BAout = 1'b0;

    PCin = 1'b0; IRin = 1'b0; MARin = 1'b0; MDRin = 1'b0; Zin = 1'b0;
    HIin = 1'b0; LOin = 1'b0; Cin = 1'b0;

    Read = 1'b0; Write = 1'b0; OutPortin = 1'b0;
    IncPC = 1'b0; CONin = 1'b0;

    ADD = 1'b0; SUB = 1'b0; AND_op = 1'b0; OR_op = 1'b0;
    SHR_op = 1'b0; SHRA_op = 1'b0; SHL_op = 1'b0; ROR_op = 1'b0; ROL_op = 1'b0;
    NEG_op = 1'b0; NOT_op = 1'b0; MUL_op = 1'b0; DIV_op = 1'b0;

    PCoutA = 1'b0; HIoutA = 1'b0; LOoutA = 1'b0; ZhighoutA = 1'b0;
    ZlowoutA = 1'b0; MDRoutA = 1'b0; InPortoutA = 1'b0; CoutA = 1'b0;

    PCoutB = 1'b0; HIoutB = 1'b0; LOoutB = 1'b0; ZhighoutB = 1'b0;
    ZlowoutB = 1'b0; MDRoutB = 1'b0; InPortoutB = 1'b0; CoutB = 1'b0;

    PCoutC = 1'b0; HIoutC = 1'b0; LOoutC = 1'b0; ZhighoutC = 1'b0;
    ZlowoutC = 1'b0; MDRoutC = 1'b0; InPortoutC = 1'b0; CoutC = 1'b0;

    case (present_state)
        S_RESET: begin
            Clear = 1'b1;
            Run    = 1'b1;
        end

        S_FETCH0: begin
            PCoutC = 1'b1;
            MARin  = 1'b1;
            IncPC  = 1'b1;
            Zin    = 1'b1;
        end

        S_FETCH1: begin
            ZlowoutC = 1'b1;
            PCin     = 1'b1;
            Read     = 1'b1;
            MDRin    = 1'b1;
        end

        S_FETCH2: begin
            MDRoutC = 1'b1;
            IRin    = 1'b1;
        end

        S_DECODE: begin
            end
    end
end

```

```

S_RR3: begin
    GrbA = 1'b1;
    RoutA = 1'b1;
    GrcB = 1'b1;
    RoutB = 1'b1;

    case (opcode)
        OP_ADD: ADD      = 1'b1;
        OP_SUB: SUB      = 1'b1;
        OP_AND: AND_op   = 1'b1;
        OP_OR:  OR_op    = 1'b1;
        OP_SHR: SHR_op   = 1'b1;
        OP_SHRA: SHRA_op = 1'b1;
        OP_SHL: SHL_op   = 1'b1;
        OP_ROR: ROR_op   = 1'b1;
        OP_ROL: ROL_op   = 1'b1;
    endcase

    Zin = 1'b1;
end

S_RR4: begin
    ZlowoutC = 1'b1;
    GraIn    = 1'b1;
    Rin      = 1'b1;
end

S_IMM3: begin
    CoutA = 1'b1;
    GrbB = 1'b1;
    RoutB = 1'b1;
    BAout = 1'b1;

    case (opcode)
        OP_LDI, OP_ADDI: ADD      = 1'b1;
        OP_ANDI:         AND_op   = 1'b1;
        OP_ORI:          OR_op    = 1'b1;
    endcase

    Zin = 1'b1;
end

S_IMM4: begin
    ZlowoutC = 1'b1;
    GraIn    = 1'b1;
    Rin      = 1'b1;
end

S_UN3: begin
    GrbB = 1'b1;
    RoutB = 1'b1;

    if (opcode == OP_NEG)
        NEG_op = 1'b1;
    else
        NOT_op = 1'b1;
    end

    Zin = 1'b1;
end

```

```

S_UN4: begin
    ZlowoutC = 1'b1;
    GraIn    = 1'b1;
    Rin      = 1'b1;
end

S_LD3: begin
    CoutA = 1'b1;

    GrbB   = 1'b1;
    RoutB   = 1'b1;

    ADD     = 1'b1;
    Zin     = 1'b1;
end

S_LD4: begin
    ZlowoutC = 1'b1;
    MARin    = 1'b1;
end

S_LD5: begin
    Read     = 1'b1;
    MDRin    = 1'b1;
end

S_LD6: begin
    MDRoutC = 1'b1;
    GraIn    = 1'b1;
    Rin      = 1'b1;
end

S_ST3: begin
    CoutA = 1'b1;

    GrbB   = 1'b1;
    RoutB   = 1'b1;

    ADD     = 1'b1;
    Zin     = 1'b1;
end

S_ST4: begin
    ZlowoutC = 1'b1;
    MARin    = 1'b1;
end

S_ST5: begin
    GraA     = 1'b1;
    RoutA     = 1'b1;
    MDRin     = 1'b1;
end

S_ST6: begin
    Write    = 1'b1;
end

S_BRANCH3: begin
    GraA     = 1'b1;
    RoutA     = 1'b1;
    CONin     = 1'b1;
end

```



```

S_BRANCH4: begin
    PCoutA = 1'b1;
    CoutB  = 1'b1;
    ADD    = 1'b1;
    Zin     = 1'b1;
end

S_BRANCH5: begin
    if (CON_FF) begin
        ZlowoutC = 1'b1;
        PCin      = 1'b1;
    end
end

S_JR3: begin
    GraC  = 1'b1;
    RoutC = 1'b1;
    PCin  = 1'b1;
end

S_JAL3: begin
    PCoutC = 1'b1;
    R12in  = 1'b1;
end

S_JAL4: begin
    GraC  = 1'b1;
    RoutC = 1'b1;
    PCin  = 1'b1;
end

S_MUL3: begin
    GraA  = 1'b1;
    RoutA = 1'b1;
    GrbB  = 1'b1;
    RoutB = 1'b1;
    MUL_op = 1'b1;
    Zin    = 1'b1;
end

S_MUL4: begin
    ZhighoutC = 1'b1;
    HIin      = 1'b1;
end

S_MUL5: begin
    ZlowoutC = 1'b1;
    LOin     = 1'b1;
end

S_DIV3: begin
    GraA  = 1'b1;
    RoutA = 1'b1;
    GrbB  = 1'b1;
    RoutB = 1'b1;
    DIV_op = 1'b1;
    Zin    = 1'b1;
end

S_DIV4: begin
    ZhighoutC = 1'b1;

```

```

        HIin      = 1'b1;
    end

    S_DIV5: begin
        ZlowoutC = 1'b1;
        LOin      = 1'b1;
    end

    S_MFHI3: begin
        HIoutC = 1'b1;
        GraIn  = 1'b1;
        Rin    = 1'b1;
    end

    S_MFLO3: begin
        LOoutC = 1'b1;
        GraIn  = 1'b1;
        Rin    = 1'b1;
    end

    S_IN3: begin
        InPortoutC = 1'b1;
        GraIn      = 1'b1;
        Rin        = 1'b1;
    end

    S_OUT3: begin
        GraC      = 1'b1;
        RoutC     = 1'b1;
        OutPortin = 1'b1;
    end

    S_HALT: begin
        Run = 1'b0;
    end
endcase
end

endmodule

```

Processor

```
`timescale 1ns/10ps

module processor (
    input wire      clock,
    input wire      clear,
    input wire [31:0] external_input,
    input wire      InPort_strobe,
    output wire [31:0] out_port_data,
    output wire      stop
);

    // =====
    // Control <-> Datapath wires
    // =====
    wire Read, Write;

    wire PCin, IRin, MARin, MDRin, Zin, HIin, LOin, Cin;
    wire GraA, GrbA, GrcA;
    wire GraB, GrbB, GrcB;
    wire GraC, GrbC, GrcC;
    wire GraIn, GrbIn, GrcIn;
    wire Rin, R12in, RoutA, RoutB, RoutC, BAout;
    wire CONin;

    wire IncPC;
    wire ADD, SUB, AND_op, OR_op;
    wire SHR_op, SHRA_op, SHL_op, ROR_op, ROL_op;
    wire NEG_op, NOT_op, MUL_op, DIV_op;

    wire PCoutA, HIoutA, LOoutA, ZhighoutA, ZlowoutA, MDRoutA, InPortoutA, CoutA;
    wire PCoutB, HIoutB, LOoutB, ZhighoutB, ZlowoutB, MDRoutB, InPortoutB, CoutB;
    wire PCoutC, HIoutC, LOoutC, ZhighoutC, ZlowoutC, MDRoutC, InPortoutC, CoutC;

    wire OutPortin;
    wire Run;
    wire Clear_from_CU;

    // Datapath observable wires
    wire [31:0] BusMuxOut_A, BusMuxOut_B, BusMuxOut_C;
    wire [31:0] PC_Q, IR_Q, HI_Q, LO_Q, Z_HI_Q, Z_LO_Q, MDR_Q, MAR_Q, InPort_Q;
    wire [31:0] R0_Q, R1_Q, R2_Q, R3_Q, R4_Q, R5_Q, R6_Q, R7_Q;
    wire [31:0] R8_Q, R9_Q, R10_Q, R11_Q, R12_Q, R13_Q, R14_Q, R15_Q;
    wire CON_FF_Q;

    // control_unit uses active-high Stop to force halt.
    // The testbench does not drive an external stop, so tie it low here.
    wire Stop_in = 1'b0;
    wire Reset = clear;

    control_unit CU (
        .Clock(clock),
        .Reset(Reset),
        .Stop(Stop_in),
        .CON_FF(CON_FF_Q),
        .IR(IR_Q),

        .Run(Run),
        .Clear(Clear_from_CU),
    );
endmodule
```

```

.GraA(GraA),
.GrbA(GrbA),
.GrcA(GrcA),
.GraB(GraB),
.GrbB(GrbB),
.GrcB(GrcB),
.GraC(GraC),
.GrbC(GrbC),
.GrcC(GrcC),
.GraIn(GraIn),
.GrbIn(GrbIn),
.GrcIn(GrcIn),
.Rin(Rin),
.R12in(R12in),
.RoutA(RoutA),
.RoutB(RoutB),
.RoutC(RoutC),
.BAout(BAout),

.PCin(PCin),
.IRin(IRin),
.MARin(MARin),
.MDRin(MDRin),
.Zin(Zin),
.HIin(HIin),
.LOin(LOin),
.Cin(Cin),

.Read(Read),
.Write(Write),
.OutPortin(OutPortin),

.IncPC(IncPC),
.CONin(CONin),

.ADD(ADD),
.SUB(SUB),
.AND_op(AND_op),
.OR_op(OR_op),
.SHR_op(SHR_op),
.SHRA_op(SHRA_op),
.SHL_op(SHL_op),
.ROR_op(ROR_op),
.ROL_op(ROL_op),
.NEG_op(NEG_op),
.NOT_op(NOT_op),
.MUL_op(MUL_op),
.DIV_op(DIV_op),

.PCoutA(PCoutA),
.HIoutA(HIoutA),
.LOoutA(LOoutA),
.ZhighoutA(ZhighoutA),
.ZlowoutA(ZlowoutA),
.MDRoutA(MDRoutA),
.InPortoutA(InPortoutA),
.CoutA(CoutA),

.PCoutB(PCoutB),
.HIoutB(HIoutB),
.LOoutB(LOoutB),

```

```

        .ZhighoutB(ZhighoutB),
        .ZlowoutB(ZlowoutB),
        .MDRoutB(MDRoutB),
        .InPortoutB(InPortoutB),
        .CoutB(CoutB),

        .PCoutC(PCoutC),
        .HIoutC(HIoutC),
        .LOoutC(LOoutC),
        .ZhighoutC(ZhighoutC),
        .ZlowoutC(ZlowoutC),
        .MDRoutC(MDRoutC),
        .InPortoutC(InPortoutC),
        .CoutC(CoutC)
    );

datapath DP (
    .clock(clock),
    .clear(clear),

    .Read(Read),
    .Write(Write),

    .PCin(PCin),
    .IRin(IRin),
    .MARin(MARin),
    .HIin(HIin),
    .LOin(LOin),
    .Zin(Zin),
    .MDRin(MDRin),
    .Cin(Cin),

    .GraA(GraA), .GrbA(GrbA), .GrcA(GrcA),
    .GraB(GraB), .GrbB(GrbB), .GrcB(GrcB),
    .GraC(GraC), .GrbC(GrbC), .GrcC(GrcC),
    .GraIn(GraIn), .GrbIn(GrbIn), .GrcIn(GrcIn),
    .Rin(Rin),
    .R12in(R12in),
    .RoutA(RoutA), .RoutB(RoutB), .RoutC(RoutC), .BAout(BAout),
    .CONin(CONin),

    .IncPC(IncPC),

    .ADD(ADD),
    .SUB(SUB),
    .AND_op(AND_op),
    .OR_op(OR_op),
    .SHR_op(SHR_op),
    .SHRA_op(SHRA_op),
    .SHL_op(SHL_op),
    .ROR_op(ROR_op),
    .ROL_op(ROL_op),
    .NEG_op(NEG_op),
    .NOT_op(NOT_op),
    .MUL_op(MUL_op),
    .DIV_op(DIV_op),

    .BusMuxOut_A(BusMuxOut_A),
    .BusMuxOut_B(BusMuxOut_B),
    .BusMuxOut_C(BusMuxOut_C),

```

```

        .PCoutA(PCoutA), .HIoutA(HIoutA), .LOoutA(LOoutA), .ZhighoutA(ZhighoutA),
.ZlowoutA(ZlowoutA), .MDRoutA(MDRoutA), .InPortoutA(InPortoutA), .CoutA(CoutA),
        .PCoutB(PCoutB), .HIoutB(HIoutB), .LOoutB(LOoutB), .ZhighoutB(ZhighoutB),
.ZlowoutB(ZlowoutB), .MDRoutB(MDRoutB), .InPortoutB(InPortoutB), .CoutB(CoutB),
        .PCoutC(PCoutC), .HIoutC(HIoutC), .LOoutC(LOoutC), .ZhighoutC(ZhighoutC),
.ZlowoutC(ZlowoutC), .MDRoutC(MDRoutC), .InPortoutC(InPortoutC), .CoutC(CoutC),

        .external_input(external_input),
        .InPort_strobe(InPort_strobe),
        .OutPortin(OutPortin),
        .OutPort_Q(out_port_data),

        .PC_Q(PC_Q),
        .IR_Q(IR_Q),
        .HI_Q(HI_Q),
        .LO_Q(LO_Q),
        .Z_HI_Q(Z_HI_Q),
        .Z_LO_Q(Z_LO_Q),
        .MDR_Q(MDR_Q),
        .MAR_Q(MAR_Q),
        .InPort_Q(InPort_Q),
        .R0_Q(R0_Q),
        .R1_Q(R1_Q),
        .R2_Q(R2_Q),
        .R3_Q(R3_Q),
        .R4_Q(R4_Q),
        .R5_Q(R5_Q),
        .R6_Q(R6_Q),
        .R7_Q(R7_Q),
        .R8_Q(R8_Q),
        .R9_Q(R9_Q),
        .R10_Q(R10_Q),
        .R11_Q(R11_Q),
        .R12_Q(R12_Q),
        .R13_Q(R13_Q),
        .R14_Q(R14_Q),
        .R15_Q(R15_Q),
        .CON_FF_Q(CON_FF_Q)
    );

    // Testbench expects stop to go high on HALT.
    assign stop = ~Run;

endmodule

```

Datapath

```
`timescale 1ns/10ps

module datapath #(parameter DATA_WIDTH = 32)
(
    input wire clock,
    input wire clear,

    input wire Read,
    input wire Write,

    // Write enables
    input wire PCin,
    input wire IRin,
    input wire MARin,
    input wire HIin,
    input wire LOin,
    input wire Zin, // enables both Z registers
    input wire MDRin,
    input wire Cin,
    input wire GraA, GrbA, GrcA, // Select and encode logic
        input wire GraB, GrbB, GrcB,
        input wire GraC, GrbC, GrcC,
        input wire GraIn, GrbIn, GrcIn,
        input wire Rin, Rl2in, RoutA, RoutB, RoutC, BAout,
        input wire CONin,

    // Special control
    input wire IncPC,

    // ALU control signals (one-hot)
    input wire ADD,
    input wire SUB,
    input wire AND_op,
    input wire OR_op,
    input wire SHR_op,
    input wire SHRA_op,
    input wire SHL_op,
    input wire ROR_op,
    input wire ROL_op,
    input wire NEG_op,
    input wire NOT_op,
    input wire MUL_op,
    input wire DIV_op,

    // 3 bus system
    output wire [DATA_WIDTH-1:0] BusMuxOut_A,
        output wire [DATA_WIDTH-1:0] BusMuxOut_B,
        output wire [DATA_WIDTH-1:0] BusMuxOut_C,

    // A bus drivers
    input wire PCoutA,
    input wire HIoutA,
    input wire LOoutA,
    input wire ZhighoutA,
    input wire ZlowoutA,
    input wire MDRoutA,
    input wire InPortoutA,
    input wire CoutA,
```

```

        // B bus drivers
input wire PCoutB,
input wire HIoutB,
input wire LOoutB,
input wire ZhighoutB,
input wire ZlowoutB,
input wire MDRoutB,
input wire InPortoutB,
input wire CoutB,

        // C bus writeback source select
input wire PCoutC,
input wire HIoutC,
input wire LOoutC,
input wire ZhighoutC,
input wire ZlowoutC,
input wire MDRoutC,
input wire InPortoutC,
input wire CoutC,

// Port specific wires
input wire [31:0] external_input,
input wire InPort_strobe,
input wire OutPortin,
output wire [31:0] OutPort_Q,

// Expose register values
output wire [DATA_WIDTH-1:0] PC_Q,
output wire [DATA_WIDTH-1:0] IR_Q,
output wire [DATA_WIDTH-1:0] HI_Q,
output wire [DATA_WIDTH-1:0] LO_Q,
output wire [DATA_WIDTH-1:0] Z_HI_Q,
output wire [DATA_WIDTH-1:0] Z_LO_Q,
output wire [DATA_WIDTH-1:0] MDR_Q,
        output wire [DATA_WIDTH-1:0] MAR_Q,
output wire [DATA_WIDTH-1:0] InPort_Q,
output wire [DATA_WIDTH-1:0] R0_Q,
output wire [DATA_WIDTH-1:0] R1_Q,
output wire [DATA_WIDTH-1:0] R2_Q,
output wire [DATA_WIDTH-1:0] R3_Q,
output wire [DATA_WIDTH-1:0] R4_Q,
output wire [DATA_WIDTH-1:0] R5_Q,
output wire [DATA_WIDTH-1:0] R6_Q,
output wire [DATA_WIDTH-1:0] R7_Q,
output wire [DATA_WIDTH-1:0] R8_Q,
output wire [DATA_WIDTH-1:0] R9_Q,
output wire [DATA_WIDTH-1:0] R10_Q,
output wire [DATA_WIDTH-1:0] R11_Q,
output wire [DATA_WIDTH-1:0] R12_Q,
output wire [DATA_WIDTH-1:0] R13_Q,
output wire [DATA_WIDTH-1:0] R14_Q,
output wire [DATA_WIDTH-1:0] R15_Q,
        output wire CON_FF_Q
);

// Select and encode logic
wire [15:0] Rin_sel;
wire [15:0] RoutA_sel, RoutB_sel, RoutC_sel;
wire [31:0] C_sign_extended;

select_encode SELECT_ENCODE (

```



```

        .IR(IR_Q),

        .GraA(GraA),
        .GrbA(GrbA),
        .GrcA(GrcA),

        .GraB(GraB),
        .GrbB(GrbB),
        .GrcB(GrcB),

        .GraC(GraC),
        .GrbC(GrbC),
        .GrcC(GrcC),

        .GraIn(GraIn),
        .GrbIn(GrbIn),
        .GrcIn(GrcIn),

        .Rin(Rin),
        .Rl2in(Rl2in),
        .RoutA(RoutA),
        .RoutB(RoutB),
        .RoutC(RoutC),
        .BAout(BAout),
        .Rin_sel(Rin_sel),
        .RoutA_sel(RoutA_sel),
        .RoutB_sel(RoutB_sel),
        .RoutC_sel(RoutC_sel),
        .C_sign_extended(C_sign_extended)
    );

    // BAout gates 0's on bus if R0 is selected
    wire [DATA_WIDTH-1:0] R0out_B;
    assign R0out_B = (BAout && RoutB_sel[0]) ? {DATA_WIDTH{1'b0}} : R0_Q;

    // Bus A control vector
    wire [23:0] bus_control_A;
    assign bus_control_A[15:0] = RoutA_sel;
    assign bus_control_A[16] = HIoutA;
    assign bus_control_A[17] = LOoutA;
    assign bus_control_A[18] = ZhighoutA;
    assign bus_control_A[19] = ZlowoutA;
    assign bus_control_A[20] = PCoutA;
    assign bus_control_A[21] = MDRoutA;
    assign bus_control_A[22] = InPortoutA;
    assign bus_control_A[23] = CoutA;

    // Bus B control vector
    wire [23:0] bus_control_B;
    assign bus_control_B[15:0] = RoutB_sel;
    assign bus_control_B[16] = HIoutB;
    assign bus_control_B[17] = LOoutB;
    assign bus_control_B[18] = ZhighoutB;
    assign bus_control_B[19] = ZlowoutB;
    assign bus_control_B[20] = PCoutB;
    assign bus_control_B[21] = MDRoutB;
    assign bus_control_B[22] = InPortoutB;
    assign bus_control_B[23] = CoutB;

    // Bus C control vector
    wire [23:0] bus_control_C;

```

```

    assign bus_control_C[15:0] = RoutC_sel;
    assign bus_control_C[16]   = HIoutC;
    assign bus_control_C[17]   = LOoutC;
    assign bus_control_C[18]   = ZhighoutC;
    assign bus_control_C[19]   = ZlowoutC;
    assign bus_control_C[20]   = PCoutC;
    assign bus_control_C[21]   = MDRoutC;
    assign bus_control_C[22]   = InPortoutC;
    assign bus_control_C[23]   = CoutC;

// Standard registers (load from BusMuxOut_C unless otherwise stated)
register IR_REG (
    .BUS_MUX_IN(IR_Q),
    .BUS_MUX_OUT(BusMuxOut_C),
    .clear(clear),
    .clock(clock),
    .enable(IRin)
);

// Only branch if CON_FF_Q is true
wire PCin_effective;
assign PCin_effective = (CONin) ? (PCin & CON_FF_Q) : PCin;
register PC_REG (
    .BUS_MUX_IN(PC_Q),
    .BUS_MUX_OUT(BusMuxOut_C),
    .clear(clear),
    .clock(clock),
    .enable(PCin_effective)
);

register HI_REG (
    .BUS_MUX_IN(HI_Q),
    .BUS_MUX_OUT(BusMuxOut_C),
    .clear(clear),
    .clock(clock),
    .enable(HIin)
);

register LO_REG (
    .BUS_MUX_IN(LO_Q),
    .BUS_MUX_OUT(BusMuxOut_C),
    .clear(clear),
    .clock(clock),
    .enable(LOin)
);

in_port INPORT_REG (
    .clk(clock),
    .clear(clear),
    .strobe(InPort_strobe),
    .external_input(external_input),
    .InPort_Q(InPort_Q)
);

out_port OUTPORT_REG (
    .clk(clock),
    .clear(clear),
    .OutPortin(OutPortin),
    .BusMuxOut(BusMuxOut_C),
    .OutPort_Q(OutPort_Q)
);

```

```

        // Internal memory subsystem wires
wire [DATA_WIDTH-1:0] BusMuxIn_MDR;
wire [DATA_WIDTH-1:0] Mdatain;
wire [8:0] mem_address;
assign mem_address = MAR_Q[8:0];
wire [DATA_WIDTH-1:0] MDR_bus_source;
assign MDR_bus_source = (Read) ? BusMuxOut_C : BusMuxOut_A;

mdr MDR_REG (
    .clk(clock),
    .clr(clear),
    .MDRin(MDRin),
    .Read(Read),
    .Mdatain(Mdatain),
    .BusMuxOut(MDR_bus_source),
    .MDR_q(MDR_Q),
    .BusMuxIn_MDR(BusMuxIn_MDR)
);

mar MAR_REG (
    .clk(clock),
    .clr(clear),
    .MARin(MARin),
    .BusMuxOut(BusMuxOut_C),
    .MAR_q(MAR_Q)
);

ram RAM (
    .clk(clock),
    .Read(Read),
    .Write(Write),
    .address(mem_address),
    .data_in(MDR_Q),
    .data_out(Mdatain)
);

// General purpose registers R0 to R15
register R0_REG(.BUS_MUX_IN(R0_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[0]));
register R1_REG(.BUS_MUX_IN(R1_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[1]));
register R2_REG(.BUS_MUX_IN(R2_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[2]));
register R3_REG(.BUS_MUX_IN(R3_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[3]));
register R4_REG(.BUS_MUX_IN(R4_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[4]));
register R5_REG(.BUS_MUX_IN(R5_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[5]));
register R6_REG(.BUS_MUX_IN(R6_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[6]));
register R7_REG(.BUS_MUX_IN(R7_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[7]));
register R8_REG(.BUS_MUX_IN(R8_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[8]));
register R9_REG(.BUS_MUX_IN(R9_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[9]));
register R10_REG(.BUS_MUX_IN(R10_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[10]));

```

```

    register R11_REG(.BUS_MUX_IN(R11_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[11]));
    register R12_REG(.BUS_MUX_IN(R12_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[12]));
    register R13_REG(.BUS_MUX_IN(R13_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[13]));
    register R14_REG(.BUS_MUX_IN(R14_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[14]));
    register R15_REG(.BUS_MUX_IN(R15_Q), .BUS_MUX_OUT(BusMuxOut_C), .clear(clear), .clock(clock),
.enable(Rin_sel[15]));

// ALU instance
wire [63:0] alu_result;

alu U_ALU (
    .A(BusMuxOut_A),
    .B(BusMuxOut_B),
    .ADD(ADD),
    .SUB(SUB),
    .AND_op(AND_op),
    .OR_op(OR_op),
    .SHR_op(SHR_op),
    .SHRA_op(SHRA_op),
    .SHL_op(SHL_op),
    .ROR_op(ROR_op),
    .ROL_op(ROL_op),
    .NEG_op(NEG_op),
    .NOT_op(NOT_op),
    .MUL_op(MUL_op),
    .DIV_op(DIV_op),
    .result(alu_result)
);

// Increment PC logic for fetch T0 (PC + 4)
wire [DATA_WIDTH-1:0] pc_plus_4;
cla #(DATA_WIDTH) U_PC_INC (
    .Z(pc_plus_4),
    .A(PC_Q),
    .Bin(32'd1),
    .sub(1'b0)
);

// What gets written into Z when Zin is asserted
wire [63:0] Z_in64 = (IncPC) ? {32'b0, pc_plus_4} : alu_result;

// Z registers load from internal Z_in64, not from BusMuxOut
register Z_LO_REG (
    .BUS_MUX_IN(Z_LO_Q),
    .BUS_MUX_OUT(Z_in64[31:0]),
    .clear(clear),
    .clock(clock),
    .enable(Zin)
);

register Z_HI_REG (
    .BUS_MUX_IN(Z_HI_Q),
    .BUS_MUX_OUT(Z_in64[63:32]),
    .clear(clear),
    .clock(clock),
    .enable(Zin)
);

```

```
// Bus mux selects which Q drives BusMuxOut
```

```
bus BusMuxA (  
    .BusOut(BusMuxOut_A),  
    .control(bus_control_A),  
    .BusIn0(R0_Q),  
    .BusIn1(R1_Q),  
    .BusIn2(R2_Q),  
    .BusIn3(R3_Q),  
    .BusIn4(R4_Q),  
    .BusIn5(R5_Q),  
    .BusIn6(R6_Q),  
    .BusIn7(R7_Q),  
    .BusIn8(R8_Q),  
    .BusIn9(R9_Q),  
    .BusIn10(R10_Q),  
    .BusIn11(R11_Q),  
    .BusIn12(R12_Q),  
    .BusIn13(R13_Q),  
    .BusIn14(R14_Q),  
    .BusIn15(R15_Q),  
    .BusIn16(HI_Q),  
    .BusIn17(LO_Q),  
    .BusIn18(Z_HI_Q),  
    .BusIn19(Z_LO_Q),  
    .BusIn20(PC_Q),  
    .BusIn21(BusMuxIn_MDR),  
    .BusIn22(InPort_Q),  
    .BusIn23(C_sign_extended)  
);
```

```
bus BusMuxB (  
    .BusOut(BusMuxOut_B),  
    .control(bus_control_B),  
    .BusIn0(R0out_B),  
    .BusIn1(R1_Q),  
    .BusIn2(R2_Q),  
    .BusIn3(R3_Q),  
    .BusIn4(R4_Q),  
    .BusIn5(R5_Q),  
    .BusIn6(R6_Q),  
    .BusIn7(R7_Q),  
    .BusIn8(R8_Q),  
    .BusIn9(R9_Q),  
    .BusIn10(R10_Q),  
    .BusIn11(R11_Q),  
    .BusIn12(R12_Q),  
    .BusIn13(R13_Q),  
    .BusIn14(R14_Q),  
    .BusIn15(R15_Q),  
    .BusIn16(HI_Q),  
    .BusIn17(LO_Q),  
    .BusIn18(Z_HI_Q),  
    .BusIn19(Z_LO_Q),  
    .BusIn20(PC_Q),  
    .BusIn21(BusMuxIn_MDR),  
    .BusIn22(InPort_Q),  
    .BusIn23(C_sign_extended)  
);
```

```
bus BusMuxC (  
    .BusOut(BusMuxOut_C),  
    .control(bus_control_C),  
    .BusIn0(R0_Q),  
    .BusIn1(R1_Q),  
    .BusIn2(R2_Q),  
    .BusIn3(R3_Q),  
    .BusIn4(R4_Q),  
    .BusIn5(R5_Q),  
    .BusIn6(R6_Q),  
    .BusIn7(R7_Q),  
    .BusIn8(R8_Q),  
    .BusIn9(R9_Q),  
    .BusIn10(R10_Q),  
    .BusIn11(R11_Q),  
    .BusIn12(R12_Q),  
    .BusIn13(R13_Q),  
    .BusIn14(R14_Q),  
    .BusIn15(R15_Q),  
    .BusIn16(HI_Q),  
    .BusIn17(LO_Q),  
    .BusIn18(Z_HI_Q),  
    .BusIn19(Z_LO_Q),  
    .BusIn20(PC_Q),  
    .BusIn21(BusMuxIn_MDR),  
    .BusIn22(InPort_Q),  
    .BusIn23(C_sign_extended)  
);
```

```

        .BusOut (BusMuxOut_C),
        .control (bus_control_C),
        .BusIn0 (R0_Q),
        .BusIn1 (R1_Q),
        .BusIn2 (R2_Q),
        .BusIn3 (R3_Q),
        .BusIn4 (R4_Q),
        .BusIn5 (R5_Q),
        .BusIn6 (R6_Q),
        .BusIn7 (R7_Q),
        .BusIn8 (R8_Q),
        .BusIn9 (R9_Q),
        .BusIn10 (R10_Q),
        .BusIn11 (R11_Q),
        .BusIn12 (R12_Q),
        .BusIn13 (R13_Q),
        .BusIn14 (R14_Q),
        .BusIn15 (R15_Q),
        .BusIn16 (HI_Q),
        .BusIn17 (LO_Q),
        .BusIn18 (Z_HI_Q),
        .BusIn19 (Z_LO_Q),
        .BusIn20 (PC_Q),
        .BusIn21 (BusMuxIn_MDR), // or MDR_Q if that is your intended registered MDR output
        .BusIn22 (InPort_Q),
        .BusIn23 (C_sign_extended)
    );

    con_ff_logic CON_FF_LOGIC (
        .clk (clock),
        .clear (clear),
        .CONin (CONin),
        .IR_c2 (IR_Q[22:19]),
        .bus_data (BusMuxOut_A),
        .CON_FF (CON_FF_Q)
    );

Endmodule

```

Select and Encode

```

module select_encode (
    input  wire [31:0] IR,

    input wire GraA,
    input wire GrbA,
    input wire GrcA,

    input wire GraB,
    input wire GrbB,
    input wire GrcB,

    input wire GraC,
    input wire GrbC,

```

```

input wire GrcC,

input wire GraIn,
input wire GrbIn,
input wire GrcIn,

input wire Rin,
input wire RoutA,
    input wire RoutB,
    input wire RoutC,
input wire BAout,

output wire [15:0] Rin_sel, // Destination register write enables
output wire [15:0] RoutA_sel, // Register output enables for A bus
output wire [15:0] RoutB_sel, // Register output enables for B bus
    output wire [15:0] RoutC_sel, // Register output enables for C bus
output wire [31:0] C_sign_extended
);

// Register fields from IR
wire [3:0] Ra, Rb, Rc;
assign Ra = IR[26:23];
assign Rb = IR[22:19];
assign Rc = IR[18:15];

// Individual decoded versions of each field
wire [15:0] decRa, decRb, decRc;
assign decRa = 16'b1 << Ra;
assign decRb = 16'b1 << Rb;
assign decRc = 16'b1 << Rc;

// Destination register select
// Whichever of Gra/Grb/Grc is asserted chooses the write destination
assign Rin_sel = Rin ?
    ({16{GraIn}} & decRa) |
    ({16{GrbIn}} & decRb) |
    ({16{GrcIn}} & decRc) : 16'b0;

```

```

// A bus source register select
// Usually Gra selects the register for A bus
assign RoutA_sel = RoutA ?
    ({16{GraA}} & decRa) |
    ({16{GrbA}} & decRb) |
    ({16{GrcA}} & decRc) : 16'b0;

// B bus source register select
// Usually Grb selects the register for B bus
assign RoutB_sel = (RoutB | BAout) ?
    ({16{GraB}} & decRa) |
    ({16{GrbB}} & decRb) |
    ({16{GrcB}} & decRc) : 16'b0;

// C bus source register select
assign RoutB_sel = (RoutB | BAout) ?
    ({16{GraB}} & decRa) |
    ({16{GrbB}} & decRb) |
    ({16{GrcB}} & decRc) : 16'b0;

// Sign-extended constant from IR[18:0]
assign C_sign_extended = {{13{IR[18]}}, IR[18:0]};

endmodule

```

Testbench

```

`timescale 1ns/10ps

module processor_tb;

    reg clock;
    reg clear;
    reg [31:0] external_input;
    reg InPort_strobe;

    wire [31:0] out_port_data;
    wire stop;

    processor DUT (
        .clock(clock),
        .clear(clear),
        .external_input(external_input),
        .InPort_strobe(InPort_strobe),
        .out_port_data(out_port_data),
        .stop(stop)
    );

```



```

task write_word;
    input [8:0] addr;
    input [31:0] data;
    integer base;
    begin
        base = addr << 2;
        DUT.DP.RAM.memory[base + 0] = data[7:0];
        DUT.DP.RAM.memory[base + 1] = data[15:8];
        DUT.DP.RAM.memory[base + 2] = data[23:16];
        DUT.DP.RAM.memory[base + 3] = data[31:24];
    end
endtask

function [31:0] read_word;
    input [8:0] addr;
    integer base;
    begin
        base = addr << 2;
        read_word = {
            DUT.DP.RAM.memory[base + 3],
            DUT.DP.RAM.memory[base + 2],
            DUT.DP.RAM.memory[base + 1],
            DUT.DP.RAM.memory[base + 0]
        };
    end
endfunction

initial begin
    clock = 1'b0;
    forever #10 clock = ~clock;
end

initial begin : run_phase3
    integer i;

    clear = 1'b1;
    external_input = 32'd0;
    InPort_strobe = 1'b0;

    for (i = 0; i < 2048; i = i + 1)
        DUT.DP.RAM.memory[i] = 8'h00;

    write_word('h000, 32'h8A800043); // ldi    R5, 0x43
    write_word('h001, 32'h8AA80006); // ldi    R5, 6(R5)
    write_word('h002, 32'h82000089); // ld     R4, 0x89
    write_word('h003, 32'h8A200004); // ldi    R4, 4(R4)
    write_word('h004, 32'h8027FFF8); // ld     R0, -8(R4)
    write_word('h005, 32'h89000004); // ldi    R2, 4
    write_word('h006, 32'h8A800087); // ldi    R5, 0x87
    write_word('h007, 32'hAA980003); // brmi   R5, 3
    write_word('h008, 32'h8AA80005); // ldi    R5, 5(R5)
    write_word('h009, 32'h80AFFFFD); // ld     R1, -3(R5)
    write_word('h00A, 32'hD0000000); // nop
    write_word('h00B, 32'hA8900002); // brpl   R1, 2
    write_word('h00C, 32'h89A80007); // ldi    R3, 7(R5)
    write_word('h00D, 32'h8B9FFFFC); // ldi    R7, -4(R3)
    write_word('h00E, 32'h03A90000); // add    R7, R5, R2
    write_word('h00F, 32'h48880003); // addi   R1, R1, 3
    write_word('h010, 32'h70880000); // neg    R1, R1
    write_word('h011, 32'h78880000); // not    R1, R1

```

```

write_word('h012, 32'h5088000F); // andi R1, R1, 0xF
write_word('h013, 32'h3A010000); // ror R4, R0, R2
write_word('h014, 32'h58A00005); // ori R1, R4, 5
write_word('h015, 32'h2A090000); // shra R4, R1, R2
write_word('h016, 32'h22A90000); // shr R5, R5, R2
write_word('h017, 32'h928000A3); // st 0xA3, R5
write_word('h018, 32'h42810000); // rol R5, R0, R2
write_word('h019, 32'h1B900000); // or R7, R2, R0
write_word('h01A, 32'h12280000); // and R4, R5, R0
write_word('h01B, 32'h93A00089); // st 0x89(R4), R7
write_word('h01C, 32'h082B8000); // sub R0, R5, R7
write_word('h01D, 32'h32290000); // shl R4, R5, R2
write_word('h01E, 32'h8B800007); // ldi R7, 7
write_word('h01F, 32'h89800019); // ldi R3, 0x19
write_word('h020, 32'h69B80000); // mul R3, R7
write_word('h021, 32'hC0800000); // mfhi R1
write_word('h022, 32'hCB000000); // mflo R6
write_word('h023, 32'h61B80000); // div R3, R7
write_word('h024, 32'h8C380002); // ldi R8, 2(R7)
write_word('h025, 32'h8C9FFFFC); // ldi R9, -4(R3)
write_word('h026, 32'h8D300003); // ldi R10, 3(R6)
write_word('h027, 32'h8D880005); // ldi R11, 5(R1)
write_word('h028, 32'h9D000000); // jal R10
write_word('h029, 32'hD8000000); // halt

write_word('h089, 32'h000000A7);
write_word('h0A3, 32'h00000068);

$display("=====");
$display("MEMORY BEFORE EXECUTION");
$display("=====");
$display("MEM[89] = %h", read_word('h089));
$display("MEM[A3] = %h", read_word('h0A3));
$display("=====");

write_word('h0B2, 32'h07450000); // add R14, R8, R10
write_word('h0B3, 32'h0ECD8000); // sub R13, R9, R11
write_word('h0B4, 32'h0F768000); // sub R14, R14, R13
write_word('h0B5, 32'hA6000000); // jr R12

#40;
clear = 1'b0;

#4000;
$display("=== MEMORY at 4000ns ===");
$display("MEM[89] = %h", read_word('h089));
$display("MEM[A3] = %h", read_word('h0A3));

wait (stop == 1'b1);

#1000;

$display("=====");
$display("PHASE 3 PROGRAM COMPLETE");
$display("=====");
$display("PC = %h", DUT.DP.PC_Q);
$display("IR = %h", DUT.DP.IR_Q);
$display("MAR = %h", DUT.DP.MAR_Q);
$display("MDR = %h", DUT.DP.MDR_Q);
$display("HI = %h", DUT.DP.HI_Q);
$display("LO = %h", DUT.DP.LO_Q);

```

```

$display("R0 = %h", DUT.DP.R0_Q);
$display("R1 = %h", DUT.DP.R1_Q);
$display("R2 = %h", DUT.DP.R2_Q);
$display("R3 = %h", DUT.DP.R3_Q);
$display("R4 = %h", DUT.DP.R4_Q);
$display("R5 = %h", DUT.DP.R5_Q);
$display("R6 = %h", DUT.DP.R6_Q);
$display("R7 = %h", DUT.DP.R7_Q);
$display("R8 = %h", DUT.DP.R8_Q);
$display("R9 = %h", DUT.DP.R9_Q);
$display("R10 = %h", DUT.DP.R10_Q);
$display("R11 = %h", DUT.DP.R11_Q);
$display("R12 = %h", DUT.DP.R12_Q);
$display("R13 = %h", DUT.DP.R13_Q);
$display("R14 = %h", DUT.DP.R14_Q);
$display("R15 = %h", DUT.DP.R15_Q);
    $display("MEM[89] = %h", read_word('h089));
$display("MEM[A3] = %h", read_word('h0A3));
$display("=====");

#100;
$stop;
end

endmodule

```